

Software Requirements Specification

Project: QR-Driven Battlepass Application

Version: 1.0.0

Date: 26 May 2026

Prepared by: Michael James Gnilo

1. Introduction

1.1 Purpose

This document outlines the functional and non-functional requirements for the "Project MakiPass" Battlepass Application. The application is designed to gamify physical and digital interactions through a tiered reward system, where progression is primarily driven by scanning location-based or event-specific QR codes.

1.2 Scope

The system will encompass a mobile client for end-users to scan codes, view missions, and claim rewards, alongside a backend administrative portal for mission management and analytics.

2. General Description

2.1 Product Perspective

The Battlepass app functions as a standalone ecosystem but may integrate with third-party authentication and payment gateways. It leverages mobile camera hardware for QR recognition and cloud-based databases for real-time progress tracking.

2.2 User Classes and Characteristics

User Class	Description
Player	End-users who scan QR codes, complete daily/weekly missions, and unlock tiers.
Administrator	Internal staff responsible for generating QR codes, setting mission parameters, and managing the reward inventory.
Moderator	Staff focused on fraud detection and verifying user-submitted completion data.

2.3 Technology Stack

The application will be developed using the following primary technologies:

- **Backend/API:** NestJS (Node.js framework)
- **Frontend/Mobile Client & Admin Dashboard:** Svelte/SvelteKit
- **Database:** Supabase (PostgreSQL)

3. System Features

3.1 QR Scanning & Mission Completion

The core mechanism involves scanning encrypted QR codes to trigger mission completion.

- **Scan Validation:** The system must verify the QR code's authenticity to prevent "quishing" or unauthorized duplication.
- **Location Awareness:** Certain missions may require GPS verification in tandem with the scan to ensure the user is physically present at

📍 Mapúa Malayan Colleges Laguna

.
- **Temporal Constraints:** Missions can be time-bound, leveraging "voltmark" concepts for spatial-temporal traceability where applicable.

3.2 Battlepass Progression System

- **XP Engine:** Users earn Experience Points (XP) for every successful scan.
- **Tier Structure:** The pass will consist of a user-defined amount of tiers, with increasing XP requirements.
- **Reward Claiming:** Upon reaching a new tier, the user can claim digital or physical rewards.

4. Functional Requirements

4.1 User Authentication

- **REQ-1:** Users must be able to create an account using email or OAuth (Google/Apple).
- **REQ-2:** The system must support Self-Sovereign Identity (SSI) for enhanced privacy in high-security environments.

4.2 Mission Management

- **REQ-3:** Admins must be able to create "Chain Missions" where a QR code that changes over time (like an authenticator OTP) is required to be flashed on a screen. The interval for a new QR code to show is 30 seconds.
- **REQ-4:** The system shall display a live countdown for limited-time event missions.

5. Non-Functional Requirements

5.1 Performance

- The QR recognition and validation process should occur within 1.5 seconds under standard 4G/5G connectivity.
- The backend should support up to 200 transactions per second (TPS) using high-throughput architectures (non-blockchain).

5.2 Security

- **Data Integrity:** All scan logs must be immutable and stored with cryptographic signatures.
- **Anti-Cheat:** The system must detect and flag anomalous scanning patterns (e.g., rapid scans from geographically distant locations).

6. External Interface Requirements

6.1 User Interface

- **Mobile App:** Optimized for iOS and Android with a "Gamified" aesthetic.
- **Admin Dashboard:** Web-based interface for mission configuration.

6.2 Hardware Interfaces

- High-resolution camera access for QR scanning.
- GPS/GNSS module for location-based missions.

7. Appendices & Placeholder Information

7.1 Stakeholder Approval

The following individuals are required to review and sign off on this document:

- **Project Lead:** Michael James Gnilo
- **Technical Lead:** Michael James Gnilo
- **Legal/Compliance:** Michael James Gnilo

7.2 Reference Documents

For further technical specifications on the QR security framework, please refer to:

- [📄 File](#) (Security Protocol)
- [📄 File](#) (UI/UX Design Specs)

7.3 Key Dates

- **Draft Finalized:**  Date
- **Development Kickoff:**  Date
- **Alpha Testing Event:**  Calendar event